



Guía de operación, respaldo y recuperación

Sistema: Tiendita Maday

Cliente: La Familia

Versión: 1.1

Fecha: 15 de julio de 2026

Audiencia: Propietario del servicio y responsable técnico

Esta guía contiene valores recomendados. Los tiempos de recuperación y retención se vuelven compromisos solo cuando se escriben y firman en el acta o contrato de soporte.

1. Arquitectura operativa

Tiendita Maday se entrega como una aplicación web de tres componentes:

Componente	Tecnología	Responsabilidad
Frontend	Angular 21.2, aplicación estática/PWA	Interfaz para clientes y personal.
Backend	Node.js y Express 4	API, permisos y reglas de negocio.
Base de datos	PostgreSQL	Datos, transacciones, auditoría y migraciones.

El despliegue previsto utiliza un proyecto Railway con servicios separados para frontend, backend y PostgreSQL. El repositorio conserva Docker Compose para desarrollo local.

2. Responsabilidades

Actividad	Propietario del negocio	Responsable técnico
Aprobar precios, roles y configuración	Responsable	Apoya
Custodiar accesos de servicios	Responsable	Administra por delegación
Despliegues y migraciones	Informado	Responsable
Respaldos y prueba de restauración	Verifica	Ejecuta
Atender alertas operativas	Responsable	Apoya en fallas técnicas
Renovar dominio y servicios	Responsable	Recuerda y verifica
Gestionar incidentes de seguridad	Decide	Contiene, investiga y documenta

3. Variables de configuración

No copie secretos a Git, documentos, tickets públicos ni mensajes sin cifrar. Configure las variables directamente en Railway.

Variable	Servicio	Sensible	Uso
DATABASE_URL	Backend	Sí	Conexión PostgreSQL.
JWT_SECRET	Backend	Sí	Firma de tokens de acceso.
JWT_REFRESH_SECRET	Backend	Sí	Firma de renovación de sesión.
ACCESS_TOKEN_EXPIRATION	Backend	No	Vigencia; por defecto 2 horas.
FRONTEND_URL	Backend	No	CORS y enlaces de correo.
BACKEND_URL	Backend	No	URL publicada en OpenAPI.
PAYPAL_CLIENT_ID / PAYPAL_SECRET	Backend	Sí	Cobro con PayPal.
PAYPAL_API_BASE	Backend	No	Sandbox o producción.
PAYPAL_CURRENCY	Backend	No	Moneda; por defecto MXN.

Variable	Servicio	Sensible	Uso
GOOGLE_CLIENT_ID	Ambos	Parcial	Inicio de sesión con Google.
CLOUDINARY_URL	Backend	Sí	Imágenes de productos.
EMAILJS_*	Backend	Sí	Verificación y recuperación por correo.
API_BASE_URL	Frontend al compilar	No	URL pública de la API con /api.

El código contiene valores de desarrollo para JWT. En producción es obligatorio definir secretos largos y aleatorios; nunca se debe operar con los valores predeterminados.

4. Despliegue inicial en Railway

4.1 Base de datos

1. Cree un proyecto Railway y agregue PostgreSQL.
2. Inicialice una instalación nueva con `Arquitectura/Database/init.sql` usando la consola de datos o `psql`.
3. Confirme que el usuario de aplicación puede conectarse.
4. Registre el propietario de la cuenta y active los controles de acceso disponibles.

4.2 Backend

1. Cree un servicio desde el repositorio.
2. Configure Root Directory como `mi cliente (la familia)/Arquitectura/Backend`.
3. Agregue las variables de la sección anterior.
4. Genere el dominio público.
5. Despliegue y compruebe `/` y `/api-docs`.

El comando de inicio ejecuta `npm run migrate` antes de levantar la API. El script aplica, en orden y una sola vez, los archivos SQL no registrados en `schema_migrations`.

4.3 Frontend

1. Cree otro servicio desde el mismo repositorio.
2. Configure Root Directory como `mi cliente (la familia)/Arquitectura/FrontEnd`.
3. Defina `API_BASE_URL` con la URL del backend y el sufijo `/api`.
4. Defina el cliente de Google si se utiliza esa autenticación.
5. Genere el dominio y compile la configuración de producción.
6. Regrese al backend y limite `FRONTEND_URL` al dominio final.

4.4 Comprobación posterior

- [] El backend responde con el mensaje de API activa.
- [] `/api-docs` abre y utiliza el servidor correcto.
- [] El frontend carga desde una ventana privada.

- [] Un administrador puede iniciar sesión.
- [] Una consulta del catálogo funciona.
- [] Los orígenes no autorizados son rechazados por CORS.
- [] El correo de verificación/recuperación llega.
- [] PayPal está en el modo esperado: sandbox o live.
- [] Cloudinary guarda y entrega una imagen de prueba.

5. Actualizaciones y migraciones

1. Revise cambios, notas de versión y respaldo reciente.
2. Ejecute pruebas y compilación en la revisión que se desplegará.
3. Registre el hash del commit.
4. Despliegue primero el backend si incluye migraciones compatibles.
5. Verifique en logs que las migraciones terminaron sin error.
6. Despliegue el frontend con API_BASE_URL correcto.
7. Ejecute la lista de comprobación posterior.
8. Conserve la revisión anterior disponible para rollback.

No interrumpa una migración. Si falla, capture el error completo y no intente marcar manualmente el archivo como aplicado.

6. Política recomendada de respaldo

Elemento	Recomendación inicial
RPO, pérdida máxima de datos	24 horas, reducir si el volumen de ventas lo exige.
RTO, tiempo objetivo de recuperación	4 horas durante horario de soporte.
Respaldo lógico PostgreSQL	Diario.
Retención	7 diarios, 4 semanales y 6 mensuales.
Cifrado	En tránsito y en reposo.
Ubicaciones	Proveedor y una copia fuera de Railway.
Prueba de restauración	Trimestral y antes de cambios de alto riesgo.
Responsable	Nombre explícito en el acta.

Los respaldos administrados del proveedor no sustituyen una copia lógica exportable bajo control del cliente.

6.1 Crear respaldo lógico

Ejecute desde un equipo seguro con PostgreSQL Client instalado:

```
pg_dump --format=custom --no-owner --no-acl --file=tiendita_YYYYMMDD_HHMM.dump
"<DATABASE_URL>"
```

Después:

1. Verifique que el archivo no esté vacío.
2. Calcule y registre una suma SHA-256.
3. Cifre o almacene el archivo en un repositorio cifrado.
4. Registre fecha, entorno, tamaño, responsable y resultado.
5. Aplique la retención sin eliminar la última copia válida mensual.

6.2 Respaldo previo a despliegue

Siempre cree un respaldo antes de migraciones, cambios masivos de catálogo o correcciones directas de datos. Etiquete la copia con el commit y motivo del cambio.

7. Restauración controlada

Restaurar reemplaza o combina datos según el destino. Nunca practique sobre producción. Primero use una base nueva y aislada.

1. Declare el incidente y detenga escrituras si existe riesgo de corrupción.
2. Seleccione el respaldo según fecha, integridad y objetivo de recuperación.
3. Cree una base PostgreSQL vacía en un entorno aislado.
4. Restaure con `pg_restore --clean --if-exists --no-owner --no-acl --dbname="<TEST_DATABASE_URL>" archivo.dump`.
5. Ejecute las migraciones de la versión de aplicación que se utilizará.
6. Compruebe usuarios, productos, inventario, pedidos, caja y totales.
7. Documente cuántas transacciones quedarían fuera del respaldo.
8. Obtenga autorización del propietario antes de sustituir producción.
9. Cambie la conexión, despliegue y ejecute pruebas de humo.
10. Conserve evidencia y realice un informe posterior al incidente.

7.1 Criterios de una restauración aprobada

- La base abre sin errores de esquema.
- Las cuentas esperadas existen y los permisos funcionan.
- Los conteos de productos, pedidos y movimientos son plausibles.
- Una venta de prueba y un ajuste de inventario se completan.
- No se utiliza el modo live de pagos durante la prueba.
- La fecha restaurada y la pérdida máxima de datos son conocidas.

8. Monitoreo e incidentes

Revise disponibilidad del frontend, salud del backend, errores HTTP, conexiones PostgreSQL, fallas de migración, correo, imágenes y pagos. Nunca incluya contraseñas, tokens ni datos completos de clientes en logs o tickets.

Severidad	Ejemplo	Respuesta inicial sugerida
Crítica	Sitio caído, cobros inconsistentes o pérdida de datos.	Contener, avisar al propietario y preservar evidencia inmediatamente.
Alta	Caja o checkout bloqueado para varios usuarios.	Diagnosticar durante horario acordado y definir alternativa operativa.
Media	Un módulo secundario falla o existe degradación.	Registrar, reproducir y programar corrección.
Baja	Consulta, texto o mejora cosmética.	Incluir en backlog.

8.1 Rollback de aplicación

Si el esquema sigue siendo compatible, vuelva a desplegar el commit anterior en Railway y verifique la API antes del frontend. Si una migración cambió datos o estructura de forma incompatible, no revierta a ciegas: restaure una copia aislada, determine impacto y obtenga autorización.

9. Transferencia y salida del proveedor

- [] El cliente es propietario o administrador de GitHub, Railway, dominio y proveedores.
- [] Se rotaron secretos compartidos durante el desarrollo.
- [] El cliente recibió un respaldo reciente y comprobó su integridad.
- [] Existe una lista de renovaciones y fechas de pago.
- [] Los canales de soporte publicados son reales.
- [] Se documentaron servicios de terceros, costos y límites.
- [] Se retiró acceso de personas que ya no requieren administración.

10. Registro de respaldos

Fecha/hora	Entorno	Archivo/ID	Tamaño	SHA-256	Responsable	Restauración probada
_____	_____	_____	_____	_____	_____	Sí / No
_____	_____	_____	_____	_____	_____	Sí / No
_____	_____	_____	_____	_____	_____	Sí / No